

Recall

A Living Memory Engine for LLM Agents

Full Strategic Analysis

Qwen Cloud Hackathon · Track 1: MemoryAgent

9 / 10

Overall verdict — unconstrained time horizon

7.5 / 10 under the 9 July 2026 hackathon deadline

Author of analysis	Claude (Anthropic)
Prepared for	Jee — Qwen Cloud Hackathon team
Submission deadline	9 July 2026
Prize pool	USD \$70,000
Track	1 — MemoryAgent
Document version	1.0 — full unabridged analysis

1. Executive Summary

Recall is a top-decile concept. The framing — **the memory engine that knows how to forget** — is not manufactured differentiation. It targets a real architectural gap that Mem0, Zep, Letta, Cognee, LangMem, and Cloudflare Agent Memory have collectively left unaddressed. The cognitive-science grounding (episodic → semantic → procedural tiers, Ebbinghaus decay, retrieval reinforcement) gives the project intellectual credibility that survives judge scrutiny.

Where the plan becomes vulnerable is execution. Eight zero-overlap features delivered to production quality, three benchmarks with charts, three framework adapters, Alibaba Cloud Terraform, a Python SDK, and a demo video — within four weeks — is a series-A engineering scope compressed into hackathon time. Without ruthless scope discipline, the submission risks looking thin across every axis rather than winning on a few.

Dimension	Score	Rationale (one line)
Idea originality	9.5/10	Decay + forgetting is a genuinely empty position in the market.
Competitive analysis	9/10	Mem0, Zep, Letta gaps are correctly diagnosed; defensible under questioning.
Qwen ecosystem fit	9.5/10	Two-tier model usage (Max + Plus) is the kind of detail engineer-judges reward.
Technical depth	9/10	Three-tier model, Bayesian confidence, provenance graphs — publishable design.
Benchmark rigor	8.5/10	Three quantitative benchmarks beat 90% of hackathon submissions that ship none.
Demo-ability	6.5/10	The core innovation is architectural — hard to show in 3 minutes of video.
Execution feasibility (4 wks)	5/10	Scope is 3–4 months of work; the plan needs cutting, not building.
Execution feasibility (open)	9/10	Unconstrained, this becomes a publishable open-source project.

Headline verdict

The **idea** deserves to win Track 1. Whether the **submission** wins depends entirely on scope discipline between now and 9 July 2026 and on how visibly the demo video can dramatize decay, conflict resolution, and provenance — the parts of Recall that exist below the UI.

2. What Recall Actually Is

Recall is an open-source memory engine that sits *above* Qwen-Agent — not next to it. Qwen-Agent provides RAG, document chunking, and basic session persistence. Recall provides the intelligence layer that decides what to remember, what to let fade, what contradicts itself, and how confident to be in each stored belief.

The system is organised as a three-tier hierarchy borrowed from human cognitive architecture, with Qwen-Max serving as the cognitive core (extraction, consolidation, conflict resolution) and Qwen-Plus serving as the operational core (retrieval-time scoring and compression).

2.1 The Three-Tier Memory Model

Tier	Contents	Lifetime	Promotion Path
1 — Episodic	Raw timestamped events from sessions — what was said, when.	Short TTL	Promoted to semantic via Qwen-Max consolidation pass.
2 — Semantic	Extracted facts, preferences, entity knowledge.	Long-lived	Patterns crystallise into procedural memory after repeated detection.
3 — Procedural	Learned how-to workflows automatically extracted from behaviour.	Near-permanent	Terminal tier — no further promotion.

2.2 Positioning Against Qwen-Agent

This framing is the single most important strategic decision in the plan and it is correctly made. Submitting a project that *extends* the Qwen ecosystem — rather than appearing to compete with Qwen-Agent — aligns Recall with the hackathon sponsor's interests and removes a category of judge objection before it can be raised.

"Qwen-Agent gives you RAG and session state. Recall is what you need above that — the system that decides what to remember, what to forget, what contradicts itself, and how confident to be about each belief. We extend Qwen. We don't replace it."

3. Strengths — Where the Plan Genuinely Wins

3.1 The Core Differentiator Is Real

Every memory tool in the market is additive. Mem0 stores facts and never removes them. Zep has temporal indexing but no passive decay. Letta requires the agent to explicitly self-edit its memory. Cognee builds knowledge graphs but treats memory as accumulation. No production system models memory as a living, decaying, self-organising substrate.

This is not a marketing gap — it is an architectural gap. The Ebbinghaus-curve decay combined with retrieval-reinforced strength is a design pattern that could be published as a standalone paper, and it is the kind of finding that wins technical track judging because it survives the question *"why hasn't anyone else done this?"* with a substantive answer rather than a hand-wave.

3.2 Competitive Analysis Holds Up Under Pressure

The plan correctly identifies what each competitor does and does not do — Mem0's static confidence, Zep's lack of decay, Letta's explicit-management requirement, Cloudflare's filter-not-optimiser retrieval. When judges probe (and they will), the answers are defensible, not improvised.

3.3 The Qwen Cost-Optimisation Pattern

Using Qwen-Max for offline consolidation and Qwen-Plus for online retrieval scoring is exactly the kind of detail engineer-judges look for. It signals production thinking — that the team has considered inference cost at scale, not just functional correctness. Most hackathon teams ignore this dimension entirely.

3.4 Three Concrete Benchmarks

Benchmark	What It Measures	Demo Value
Long-horizon recall accuracy	Recall@K across 50 multi-session synthetic conversations. Compared against no-memory baseline and Mem0.	Quantitative — shows superiority on a metric judges trust.
Context efficiency	Tokens consumed vs. information coverage delivered.	Demonstrates that budget-aware retrieval beats naive top-K.
Forgetting curve validation	Memory strength over time, with and without retrieval reinforcement.	Visually compelling — the only chart that shows decay <i>happening</i> .

Most hackathon submissions ship with zero quantitative claims. Three benchmarks with comparative charts puts Recall in the top 5–10% of submissions purely on rigour.

3.5 Production-Grade Repo Structure

The proposed repo layout — separate modules for ingestion, consolidation, decay, conflict, provenance, prefetch, retrieval, plus integrations and examples folders — signals that this is not a one-file demo. The presence of **integrations/** and **examples/** communicates intent to be adopted, not just judged. That perception alone differentiates from the median submission.

4. Weaknesses — Where the Plan Is Vulnerable

4.1 Scope Is the Single Biggest Threat

Count the deliverables: three storage tiers, five core modules, conflict detection, provenance graphs, intent classifier with warm cache, budget-aware retrieval, REST API, Python SDK, three framework adapters (LangChain, LangGraph, AutoGen), three benchmarks with charts, Terraform infrastructure, architecture diagram, and a 3-minute demo video — in four weeks.

A realistic ship-list for one engineer in four weeks is approximately 30–40% of the above. The plan as written is a 3–4 month engineering project. Without scope cuts, two outcomes are likely: either the submission ships with stubs and broken integrations, or the core engine is rushed and fails the benchmark. Both lose. The plan needs cutting, not building.

Critical scope risk

Feature 3 (predictive pre-fetching) is already marked *"if time permits"* — yet it is also sold as one of eight zero-overlap differentiators. If it does not ship, the claim weakens. If it does ship at low quality, the demo embarrasses the concept. Either way, this is a feature to **cut from the headline narrative** rather than defer.

4.2 The Demo-ability Problem

Memory systems do not demo well. Decay is invisible. Provenance graphs are static structures. Bayesian confidence updates are numerical changes in a database. A chatbot can be filmed in 30 seconds and immediately understood. Recall cannot.

This is the single largest reason a brilliant idea can lose to a less-original idea with better visual presentation. The plan must explicitly solve the demo problem — for example, a split-screen interface that shows the memory store mutating live while the agent converses, or a 30-day fast-forward animation of decay reshaping the memory tier composition.

4.3 Procedural Memory Extraction Is Quietly Hard

The plan claims procedural memory "automatically crystallises" repeated workflows after detection. In practice, pattern detection across heterogeneous sessions is one of the hardest problems in the system. A misfiring classifier will produce false procedural memories that mislead the agent, creating exactly the hallucination problem Recall is supposed to solve. This module is more fragile than it appears in the plan.

4.4 Eight Features Is a Pitch-Deck Number, Not a Hackathon Number

Hackathon judging rewards **one breathtaking feature executed perfectly over eight features at 60% completion**. The current framing — eight zero-overlap differentiators in a competitive comparison table — reads as Series-A pitch material rather than a hackathon entry. This is a pitch strategy problem, not a product problem.

4.5 Provenance Graphs Need a Real Storage Choice

The plan describes a provenance graph but does not commit to a graph database. Implementing graph traversal on top of Postgres works for a demo, but the architecture document needs to either (a) name a graph store such as Neo4j or Nebula Graph, or (b) explicitly justify the relational implementation as a deliberate simplification. Judges who care about provenance will ask.

5. Feature-by-Feature Evaluation

Each of the eight claimed differentiators evaluated on novelty, technical risk, demo value, and verdict on whether to keep it in the 4-week submission.

Feature	Novelty	Tech Risk	Demo Value	Keep / Cut
1. Mathematical decay with retrieval reinforcement The headline differentiator. Ebbinghaus curve + per-tier rates + soft-delete tombstones.	Very High	Medium	High (chart)	KEEP — core
2. Automated procedural extraction Passive detection of repeated workflows across sessions, then crystallisation.	High	High	Medium	SCOPE DOWN — ship simple pattern matcher, not full classifier.
3. Context budget as first-class API parameter Scoring + packing algorithm vs. simple filter.	High	Low	High (chart)	KEEP — core
4. Conflict detection + resolution Semantic check on ingestion; auto-resolve or flag.	Very High	Medium	Very High	KEEP — headline
5. Provenance graph with confidence evolution Queryable derivation history per memory.	High	Medium	High	KEEP — simplified
6. Predictive pre-fetching Intent classifier on last 3 turns; warm cache.	Medium	High	Low	CUT from 4-wk plan; revisit post-hackathon.
7. Bayesian confidence accumulation Four-rule update model with crystallisation.	High	Low	Medium	KEEP — cheap to implement
8. Cross-agent memory scoping private / team / global / user-owned permission layer.	Medium	Low	Low	CUT unless multi-agent demo is the headline.

Recommended 4-week scope

Ship features **1, 3, 4, 5 (simplified), and 7** to 95% completion with strong benchmarks. Feature 2 ships as a pattern-matcher demo. Features 6 and 8 cut entirely from the submission and documented as roadmap. This becomes the strongest possible Track 1 entry.

6. Competitive Landscape

An honest mapping of where Recall sits against existing tools. The market is crowded with memory layers; what is not crowded is the specific position of *passive forgetting* + *conflict resolution* + *provenance*. That triple is genuinely empty.

Capability	Mem0	Zep	Letta	Cognee	Cloudflare AM	Recall
Passive decay	✗	✗	✗	✗	✗	✓
Retrieval reinforcement	✗	Partial	✗	✗	✗	✓
Conflict detection	✗	✗	Manual	✗	✗	✓
Provenance graph	✗	✗	✗	Partial	✗	✓
Budget-aware retrieval	✗	✗	✗	✗	Filter only	Packer
Three-tier model	✗	✗	Partial	✗	✗	✓
Procedural extraction	✗	✗	Manual	✗	✗	Auto
Bayesian confidence	Static	✗	✗	✗	✗	Dynamic

The honest reading: **no single competitor matches more than two of Recall's eight capabilities**. The cumulative gap is genuine. Where the table risks overselling is in collapsing partial implementations to ✗ — Zep's temporal indexing is closer to passive decay than the cell suggests. Acknowledge this proactively in the submission to look more rigorous, not less.

7. Architecture Review

7.1 The Five-Module Decomposition

Module	Purpose	Critical Implementation Risk
Ingestion	Qwen-Max extractor → structured memory units → dedup → write.	Extraction prompt design; structured-output reliability.
Consolidation	Scheduled Qwen-Max pass — merge duplicates, promote tiers, detect patterns.	Pattern detection logic; merge conflict heuristics.
Decay engine	Per-tier decay curves, reinforcement on retrieval, soft-delete tombstoning.	Tuning decay constants; threshold calibration.
Conflict engine	Semantic conflict check; auto-resolve or flag; conflict history.	False-positive rate; resolution prompt brittleness.
Retrieval	Score by relevance × recency × strength; pack within token budget.	Knapsack-style packing; compression-quality tradeoff.

7.2 Alibaba Cloud Stack — Assessment

Component	Service Selected	Verdict
Heavy LLM	Qwen-Max	Correct — quality matters for consolidation.
Light LLM	Qwen-Plus	Correct — latency matters for retrieval scoring.
Vector store	Milvus on Lindorm	Native to Alibaba Cloud — judge-pleasing.
Structured store	ApsaraDB PostgreSQL	Works, but graph queries will hurt — consider a graph DB add-on.
Compute	ECS or ACK	Either works; ACK if you want Kubernetes story.
Scheduled jobs	Function Compute	Right call for consolidation cron.
Observability	Langfuse	Familiar to you; documents the LLM call graph.

7.3 Architectural Gaps to Address Before Submission

- **Graph storage commitment.** Decide explicitly between Neo4j, Nebula Graph, or a relational implementation with documented limits. The current plan leaves this open.
- **Soft-delete reclamation policy.** Tombstones grow unboundedly. The architecture document needs a vacuum/compaction schedule or audit-only retention window.
- **Embedding model choice.** Vector search is mentioned but the embedding model is not named. Qwen-Embedding or text-embedding-v3 (DashScope) should be specified.
- **Multi-tenancy boundary.** If memory is per-user, the schema needs explicit tenant_id columns and isolation guarantees — important if any judge asks about production deployment.

- **Failure mode for Qwen-Max outages.** What happens to ingestion if the consolidation API is unavailable? A queue-and-retry pattern needs to be documented.

8. Risk Matrix

Risk	Likelihood	Impact	Mitigation
Scope overrun by 9 July	High	Critical	Cut to 5 features; freeze scope by end of Week 1.
Demo video fails to show decay visually	Medium	Critical	Build a dedicated memory-tier visualisation panel before any other UI work.
Procedural extraction misfires in benchmark	Medium	High	Ship simple recurrence detection; document classifier as future work.
Conflict resolution prompt is brittle	Medium	High	Test on 30+ synthetic contradiction pairs; include eval in repo.
Benchmark dataset is too synthetic to convince judges	Medium	Medium	Augment with at least one real-world conversational dataset (e.g. LongMemEval).
Alibaba Cloud Terraform debugging eats Week 3	Low–Medium	High	Provision infrastructure in Week 1; treat deploy as a Week 1 deliverable, not Week 3.
Judges miss the differentiator in 3-min video	Medium	Critical	Open the video with a 15-sec animated decay curve; never bury the lede.
Mem0 / Zep ships passive decay before 9 July	Low	Critical	Monitor competitor releases weekly; have a pivot to provenance-led positioning ready.

9. Recommended Submission Scope

9.1 The Ruthless Cut

The submission becomes stronger, not weaker, by shipping less. The argument: judges evaluate depth × clarity × differentiation, not raw feature count. A focused entry that proves three things rigorously beats a sprawling entry that demonstrates eight things partially.

Decision	Items	Rationale
SHIP TO 95%	Decay engine · Conflict detection · Budget-aware retrieval · Bayesian confidence · Three-tier storage · REST API · Python SDK · LangGraph adapter only	These are the irreducible core. The decay + conflict + provenance triple is the genuine differentiator. LangGraph is your existing stack.
SHIP SIMPLIFIED	Provenance graph (relational impl) · Procedural extraction (pattern matcher, not classifier)	Demonstrate the concept; document the path to full implementation as roadmap.
CUT	Predictive pre-fetching · Cross-agent scoping · LangChain adapter · AutoGen adapter · Customer-support example	Each carries 3–7 days of implementation and adds little to the headline narrative.

9.2 Revised Three-Week Build Plan

Week	Focus	Outcome
Week 1 Now → +7 days	Alibaba Cloud infra · three-tier storage schema · Qwen-Max extractor · REST API skeleton · decay engine math	End-to-end <i>thin</i> path: session in, memory out, decay running.
Week 2 +7 → +14 days	Conflict engine · simplified provenance · Bayesian confidence rules · budget-aware retrieval packer · LangGraph adapter	All five core differentiators functional. First benchmark numbers.
Week 3 +14 → +21 days	Three benchmarks complete · charts · README · architecture diagram · demo video (memory-tier visualisation panel) · examples folder	Submission-ready artifact.
Week 4 (buffer) +21 → 9 July	Polish · re-record video · benchmark re-runs · documentation hardening	Slack for unexpected debugging. Do not add features here.

10. Demo & Submission Strategy

10.1 The Demo Video — Where Most of Recall Will Be Won or Lost

Track 1 judges will spend approximately 60–90 seconds on the average submission before deciding whether to keep watching. The first 15 seconds must dramatize the differentiator. The plan as written does not solve this problem.

Proposed 3-minute video structure:

Time	Content	Purpose
00:00–00:15	Animated forgetting curve. Two memory stores side by side — Mem0 vs. Recall — after 6 months of use. Mem0 is bloated; Recall is clean.	Hook. Differentiator visible in 15 seconds.
00:15–00:45	Voice-over: the problem. Agent confidently hallucinates from a 4-month-old preference. Cut to Recall correctly recognising it as stale.	Problem statement + solution preview.
00:45–01:30	Architecture diagram fly-through. Three tiers, decay engine, conflict resolver, Qwen-Max + Qwen-Plus split.	Establish technical credibility.
01:30–02:30	Live demo. Agent conversation in left panel. Memory tier visualisation in right panel — see memories appear, promote, fade, get resolved when contradictory.	Make the invisible visible.
02:30–03:00	Three benchmark charts in rapid sequence. Closing line: <i>"This is what the Qwen ecosystem needs next."</i>	Rigour + sponsor-aligned close.

10.2 README — The Other Submission Artifact

Judges read repos. The README needs three things in the first screen: a one-line pitch, an architecture diagram, and a one-command quickstart against Alibaba Cloud. The current plan does not specify quickstart latency — aim for under 5 minutes from clone to running benchmark.

10.3 Positioning Statement — Final Form

"Recall is the first open-source memory engine for LLM agents that models memory as a living system — where memories decay, reinforce, and self-organise rather than accumulate indefinitely. Built on Qwen-Max for consolidation and Qwen-Plus for retrieval, deployable on Alibaba Cloud in one command, integrated with LangGraph out of the box. The system Qwen-Agent needs above it."

11. Final Verdict

Unconstrained	Hackathon-bounded
9 / 10	7.5 / 10
With time and resources, this becomes a publishable open-source memory engine.	As written, the scope risks turning a 9 idea into a 7.5 submission.

11.1 What Wins This Hackathon

- **Scope down to 5 features.** Ship them to 95% completion. Document the rest as roadmap.
- **Solve the demo problem first, not last.** Build the memory-tier visualisation panel in Week 1, not Week 3. It is the single highest-leverage UI work in the entire project.
- **Three benchmarks beat eight features.** Make every benchmark produce a chart, and put all three in the README above the fold.
- **Open the demo video with 15 seconds of decay.** Never bury the lede. Judges decide in the first quarter of the runtime.
- **Position as Qwen ecosystem extension, not Mem0 killer.** The current framing already does this — protect it.

11.2 The Honest Closing

Recall is among the strongest Track 1 ideas the Qwen Cloud Hackathon will see. The cognitive-science foundation, the architectural gap it targets, the Qwen ecosystem positioning, and the benchmark rigour are all top-decile. The differentiation is real and survives scrutiny.

The only thing standing between this plan and a top-three Track 1 finish is scope discipline and demo presentation. Both are solvable. Neither is automatic. The next decision is not whether to build Recall — it is what to **not** build between now and 9 July 2026.

Closing assertion

Build the decay engine, the conflict resolver, the provenance graph, the budget-aware retriever, the Bayesian confidence rules, and one framework adapter. Cut everything else from the submission. Make the demo video show the memory store mutating live. That is the winning entry.